🛒 SHOPEZ: E-Commerce

Project Description:

SHOPEZ is a full-stack e-commerce web application designed to provide users with a seamless online shopping experience. The platform allows customers to browse products, add items to cart, securely place orders, and track deliveries. Vendors can manage their product listings, while administrators oversee the entire system, including users, orders, and inventory. The application is built using modern technologies such as React (frontend), Node.js with Express (backend), and MongoDB (database), ensuring scalability, security, and high performance.

Scenario:

A user visits SHOPEZ to purchase products online. After registering and logging in, the user browses products using search and filters. The user selects a product, adds it to the cart, and proceeds to checkout using a preferred payment method. Once the order is placed, the user can track delivery and provide feedback. Meanwhile, vendors manage product listings and inventory, and the admin monitors system activities, handles disputes, and ensures smooth operations.

TECHNICAL ARCHITECTURE

🧱 1. Frontend Layer (Client Side)

The frontend is the **user-facing part** of the application. It is responsible for rendering UI components and handling user interactions.

🔧 Technologies Used

- React
- Vite (for fast build and development)

🎯 Responsibilities

- Displays product listings, categories, and details
- Handles user actions (login, add to cart, checkout)
- Manages client-side routing

- Sends and receives data from backend APIs

⚙️ Key Components

- **Home Page –** Displays featured products
- **Product Page –** Shows product details
- **Cart Page –** Manages selected items
- **Authentication Pages –** Login & Signup
- **Order Page –** Displays order history

🔄 Working

When a user performs an action (e.g., clicking "Add to Cart"):

1. React captures the event
2. Sends an HTTP request (GET/POST) to backend API
3. Waits for response
4. Updates UI dynamically (without page reload)

⚙️ 2. Backend Layer (Server Side)

The backend acts as the **brain of the application**, handling logic, processing requests, and managing communication between frontend and database.

🔧 Technologies Used

- Node.js
- Express.js

🎯 Responsibilities

- Handles REST API requests
- Implements business logic

- Authenticates users
- Manages orders and payments
- Interacts with database

🧩 Core Modules

1. User Module

- Registration and login
- Profile management
- Password encryption

2. Product Module

- Add/update/delete products
- Fetch product listings
- Category filtering

3. Cart Module

- Add/remove items
- Update quantities

4. Order Module

- Create orders
- Track order history

🔐 Authentication & Security

- Uses JSON Web Token (JWT)
- Passwords stored in encrypted format
- Secure API endpoints

🔄 API Communication

- Uses REST APIs:
  - GET /products
  - POST /login
  - POST /orders
  - PUT /cart

🗄️ 3. Database Layer

The database layer is responsible for **storing and retrieving application data**.

🔧 Technology Used

- MongoDB

📦 Collections (Tables Equivalent)

1. Users Collection

- User ID
- Name
- Email
- Password (hashed)

2. Products Collection

- Product ID
- Name
- Price
- Description
- Stock

3. Orders Collection

- Order ID
- User ID

- Product List
- Total Amount
- Status

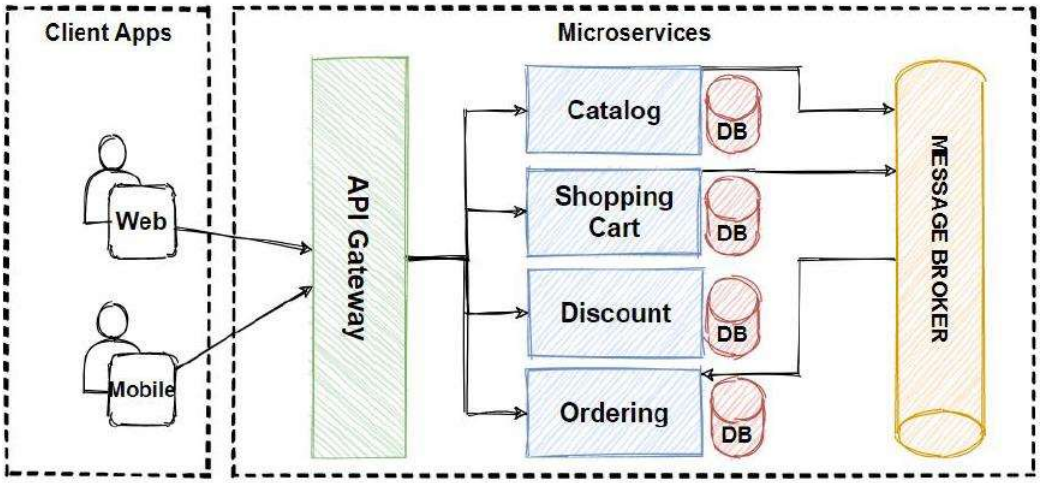
4. Cart Collection

- User ID
- Items
- Quantity

⚙️ **Database Operations**

- Insert (create new user/order)
- Read (fetch products)
- Update (modify cart/order)
- Delete (remove items)

- 📊 **Architecture Diagram**



ER Diagram :

🧩 Entities and Attributes

1. 👤 User Entity

Represents customers and administrators of the system.

Attributes:

- **User_ID (Primary Key)** – Unique identifier
- Name – Full name of the user
- Email – Unique email address
- Password – Encrypted password
- Role – Defines access (Admin/User)

---

2. 🛍️ Product Entity

Represents items available for purchase.

Attributes:

- **Product_ID (Primary Key)**
- Name – Product name
- Price – Product price
- Category – Product type
- Stock – Available quantity
- Description – Product details

---

3. 🛒 Cart Entity

Represents the temporary storage of selected items before purchase.

Attributes:

- **Cart_ID (Primary Key)**
- **User_ID (Foreign Key)**
- Products – List of selected items
- Quantity – Number of items

---

4. 📦 Order Entity

Represents completed purchases.

Attributes:

- **Order_ID (Primary Key)**
- **User_ID (Foreign Key)**
- **Product_ID (Foreign Key)**
- Quantity – Number of products
- Total_Price – Total cost
- Status – (Pending, Shipped, Delivered)

---

5. 💳 Payment Entity

Represents payment details for orders.

Attributes:

- **Payment_ID (Primary Key)**
- **Order_ID (Foreign Key)**
- Method – (UPI, Card, COD)
- Status – (Success, Failed)

🔗 Relationships Between Entities

1. User → Order (One-to-Many)

- One user can place multiple orders
- Each order belongs to one user

👉 Example:
A customer can order multiple products at different times

---

2. User → Cart (One-to-One)

- One user has one cart
- Each cart belongs to one user

👉 The cart is unique per user

---

3. Order → Payment (One-to-One)

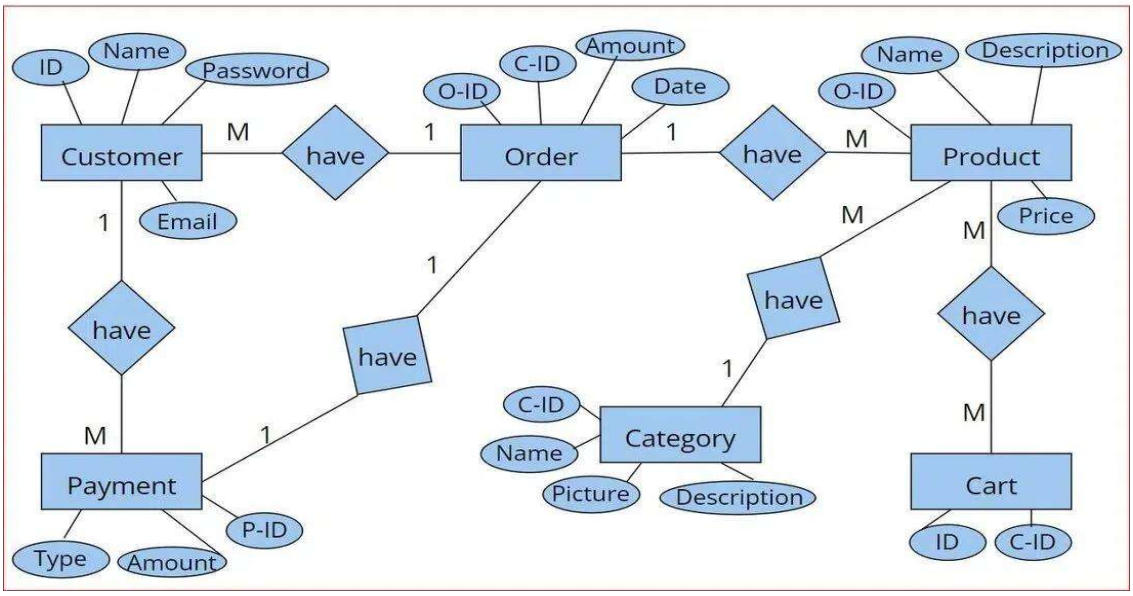
- Each order has one payment
- Each payment is linked to one order

---

4. Product → Order (One-to-Many)

- One product can appear in multiple orders
- Each order references a product

ER DIAGRAM :



Features

1. 🔐 User Authentication (JWT-Based Login/Signup)

The system implements secure authentication using **JSON Web Token (JWT)**.

🔧 Features:

- User registration (Signup)
- User login with email and password
- Token-based authentication
- Secure password storage (hashed)

⚙️ Working:

- On login, the backend generates a JWT token
- The token is stored on the client side
- Each request includes the token for verification
- Unauthorized users are denied access

🎯 Benefits:

- Stateless authentication
- Improved security
- Scalable for large systems

2. 🛍️ Product Browsing and Search

Users can explore available products easily.

🔧 **Features:**

- View all products
- Category-based filtering
- Keyword-based search
- Product detail pages

⚙️ **Working:**

- Frontend sends search queries to backend
- Backend fetches filtered data from database
- Results are displayed dynamically

3. 🛒 Add to Cart & Wishlist

Allows users to manage desired products before purchase.

🔧 **Features:**

- Add/remove items from cart
- Update product quantity
- Save items to wishlist

⚙️ **Working:**

- Cart data is stored in database
- Wishlist stores favorite items for later

🎯 **Benefits:**

- Enhances user experience
- Encourages repeat visits

4. 💳 Secure Checkout

Handles the final purchase process securely.

🔧 **Features:**

- Address selection
- Order summary
- Payment selection

⚙️ **Working:**

- Cart items are validated
- Order is created in backend
- Payment process is initiated

5. 📦 Order Tracking

Allows users to monitor their orders.

🔧 **Features:**

- View order history
- Track order status (Pending, Shipped, Delivered)
- Real-time updates

🎯 **Benefits:**

- Improves transparency
- Builds user trust

6. 💰 Payment Integration

Integrates with external payment services.

🔧 **Features:**

- Multiple payment methods (UPI, Card, COD)
- Payment status tracking
- Secure transactions

⚙️ **Working:**

- User selects payment method
- Redirect to payment gateway
- Response updates order status

7. 👨‍💼 Admin Dashboard

Provides control over the entire system.

🔧 **Features:**

- Manage users
- View orders
- Monitor sales
- Manage products

🎯 **Benefits:**

- Centralized control
- Efficient system monitoring

8. 🏪 Vendor Product Management

Allows vendors to manage their products.

🔧 **Features:**

- Add new products
- Update product details
- Delete products

- Manage inventory

🎯 **Benefits:**

- Supports multi-vendor system
- Simplifies product handling

9. 📱 Responsive UI (Tailwind CSS / Bootstrap)

Ensures the application works on all devices.

🔧 **Technologies:**

- Tailwind CSS
- Bootstrap

🎯 **Features:**

- Mobile-friendly design
- Adaptive layouts
- Smooth user experience

10. 🔒 Protected Routes

Restricts access to authorized users only.

🔧 **Features:**

- Role-based access control (Admin/User)
- Private routes for dashboard and orders
- Token validation

⚙️ **Working:**

- Routes are checked for valid JWT
- Unauthorized users are redirected to login

Roles and Responsibilities

1. Customer (End User)

The customer is the primary user of the platform who interacts with the system to purchase products.

🔧 Responsibilities

1. Registration and Login

- Customers can create an account using email and password
- Secure login using **JSON Web Token (JWT)**
- Maintain personal profile information

---

2. Browse Products

- View available products
- Filter products by category
- Search products using keywords
- View detailed product information

---

3. Add to Cart

- Add products to cart
- Update quantity of items
- Remove unwanted items

---

4. Place Orders

- Proceed to checkout
- Provide shipping details

- Select payment method
- Confirm order

---

5. Track Orders

- View order history
- Check order status (Pending, Shipped, Delivered)
- Receive notifications (optional)

🎯 Importance

Customers drive the platform's revenue by purchasing products and interacting with the system.

🏪 2. Vendor (Seller)

Vendors are responsible for managing and selling products on the platform.

🔧 Responsibilities

1. Product Management

- Add new products
- Edit product details (price, description, stock)
- Delete products when necessary

---

2. Inventory Management

- Monitor stock levels
- Update product availability
- Prevent out-of-stock issues

---

3. Order Management

- View incoming orders
- Process and prepare shipments
- Update order status

🎯 Importance

Vendors ensure product availability and quality, making the platform functional and attractive to customers.

👨‍💼 3. Admin (System Administrator)

The admin has full control over the system and ensures proper functioning.

🔧 Responsibilities

1. User and Vendor Management

- Add/remove users
- Approve or block vendors
- Manage user roles and permissions

---

2. Product Approval

- Review products added by vendors
- Approve or reject listings
- Ensure quality and compliance

---

3. System Monitoring

- Monitor system performance
- Track orders and transactions
- Analyze system usage

---

4. Complaints and Refund Handling

- Handle customer complaints
- Process refund requests
- Resolve disputes between users and vendors

User Flow

1. 🌐 User Visits Website

- The user opens the SHOPEZ website using a browser
- The homepage loads with featured products, categories, and offers
- Navigation menu allows access to login, cart, and product sections

---

2. 🔐 Registration / Login

- New users create an account by providing:
  - Name
  - Email
  - Password
- Existing users log in using credentials
- Authentication is handled using **JSON Web Token (JWT)**

🔄 Process:

- Credentials sent to backend
- Backend validates user
- Token is generated and stored
- User is redirected to homepage/dashboard

---

3. 🔍 Browse/Search Products

- User explores available products
- Can filter by:
  - Category

    - Price
    - Ratings (optional)
- Uses search bar for quick product lookup

⚙️ Behind the scenes:

- Frontend sends query to backend
- Backend fetches data from database
- Results displayed instantly

---

4. 📄 View Product Details

- User clicks on a product
- Detailed page shows:
    - Product name
    - Price
    - Description
    - Stock availability
    - Images

---

5. 🛒 Add to Cart

- User selects quantity
- Clicks “Add to Cart”

⚙️ Process:

- Request sent to backend
- Product added to user’s cart in database
- Cart UI updates instantly

---

6. 💳 Proceed to Checkout

- User opens cart

- Reviews selected items
- Clicks “Checkout”

🔧 Actions:

- Enters shipping address
- Confirms order summary
- Selects payment method

---

7. 💰 Makes Payment

- User chooses payment option:
  - UPI
  - Credit/Debit Card
  - Cash on Delivery

⚙️ Process:

- Payment request sent to gateway
- Transaction is processed securely
- Backend receives payment status

---

8. ✅ Order Confirmation

- After successful payment:
  - Order is created in database
  - Confirmation message is displayed

📩 Output:

- Order ID generated
- Confirmation shown on UI
- Optional email/SMS notification

---

9. 📦 Track Order

- User navigates to “My Orders”
- Can view:
    - Order status (Pending, Shipped, Delivered)
    - Order details

---

10. ⭐ Give Feedback

- After delivery, user can:
    - Rate the product
    - Write a review

🎯 Purpose:

- Helps other users
- Improves product quality
- Provides insights to vendors

USER FLOW :

User → Visit Website

↓

Register / Login

↓

Browse / Search Products

↓

View Product Details

↓

Add to Cart

↓

Checkout

↓

Payment

↓

Order Confirmation

↓

Track Order

↓

Give Feedback

MVC Pattern (Model–View–Controller Architecture)

1. Model Layer (Data Layer)

The Model layer represents the **data structure and database logic** of the application.

🔧 Technologies Used

- MongoDB
- Mongoose

🎯 Responsibilities

- Define database schemas
- Handle CRUD operations (Create, Read, Update, Delete)
- Validate data before storing
- Interact directly with the database

📦 Example Models

👤 User Model

- Stores user details (name, email, password, role)
- Handles authentication-related data

🛍️ Product Model

- Stores product information (name, price, category, stock)

📦 Order Model

- Stores order details (user, products, total price, status)

2. Controller Layer (Business Logic Layer)

The Controller layer acts as the **brain of the application**, handling logic and processing requests.

🔧 Technologies Used

- Node.js
- Express.js

🎯 Responsibilities

- Receive requests from routes
- Process business logic
- Interact with models
- Send responses back to client

🧩 Example Controllers

🔐 Login Controller

- Validates user credentials
- Generates JWT token

🛍️ Product Controller

- Fetches product list
- Adds/updates products

📦 Order Controller

- Creates new orders

- Updates order status

⚙️ Example Flow

1. Request comes from frontend
2. Controller processes request
3. Calls model to access data
4. Sends response (JSON) back

🌐 3. View Layer (Routing Layer in Backend)

In SHOPEZ backend, the "View" is implemented as **API routes** (since frontend is separate).

🔧 Technologies Used

- Express.js

🎯 Responsibilities

- Define API endpoints
- Route requests to appropriate controllers
- Act as a bridge between frontend and backend logic

📍 Example Routes

- /api/auth → Authentication routes
- /api/products → Product routes
- /api/orders → Order routes